

Technical Assignment - Software Engineer

Context

We build SaaS platforms and automation tools for financial-services teams. Data may arrive in structured files such as CSV, but also in invoices, bank statements and other PDF documents.

The goal is to build a small application that imports, extracts, validates, corrects and approves financial records from both CSV and PDF sources.

You have **two calendar days** to submit the assignment. We recommend limiting the effective work to approximately **8 to 10 hours**. We value prioritization and clarity more than feature count.

Provided files

- `transactions_import.csv`
- `invoice_legal_services.pdf`
- `invoice_software_subscription.pdf`
- `bank_statement_july_2026.pdf`
- `Data_Dictionary.pdf`

Main workflow

A user should be able to:

1. Create an import batch.
2. Upload either a CSV file or one or more PDF documents.
3. Extract and normalize records into the common data model.
4. View valid records and records requiring review.
5. See field-level validation errors.
6. Correct extracted or imported values.
7. Re-run validation after a correction.
8. Validate an individual record.
9. View a batch summary.

CSV ingestion

The CSV contains both valid and intentionally invalid rows. Import all rows rather than rejecting the whole file.

The application should preserve the original source filename and associate each record with its import batch.

PDF extraction through an AI provider

Implement a connector to one AI provider capable of reading PDF documents or PDF-derived content.

Examples include OpenAI, Anthropic, Google Gemini, Azure OpenAI or another provider of your choice.

Requirements:

- Provider credentials must be read from environment variables.
- The provider implementation should be isolated behind a clear interface or service.
- PDF content must be converted into structured output matching the common data model.
- The extracted response must be validated before persistence.
- The application must handle provider errors, invalid JSON or incomplete extraction without crashing.
- Records with missing required fields or low confidence must be saved with status `NEEDS_REVIEW`.
- The README must explain the provider choice and how to configure it.
- Do not commit API keys.

A real provider call is expected. If access to a paid provider is genuinely impossible, provide a mock implementation and clearly document how the real connector would work. A real integration will be evaluated more positively.

The PDFs are intentionally different:

- Two supplier invoices, each expected to produce one normalized record.
- One bank statement, expected to produce multiple transaction records.

Validation rules

Use the rules in `Data_Dictionary_EN.docx`.

Validation must be performed server-side even if the frontend also validates inputs.

A record can move through the following statuses:

- `NEEDS_REVIEW`
- `VALID`
- `VALIDATED`

A corrected record should be revalidated before it can become `VALIDATED`.

Backend

Create an API supporting at least:

- Create an import batch
- Upload a CSV
- Upload one or more PDFs
- List import batches
- Retrieve records for a batch
- Retrieve validation errors for a record
- Edit a record
- Re-run validation
- Validate a record

- Retrieve a batch summary

The exact endpoint design is up to you.

Frontend

Create a clear interface supporting:

- Batch creation or selection
- CSV and PDF upload
- Processing status
- Record list
- Filtering by source and status
- Field-level errors
- Editing and revalidation
- Individual validation
- Batch summary

The design does not need to be elaborate, but the workflow should be understandable and usable.

Recommended stack

- Python
- FastAPI
- PostgreSQL
- React
- Docker

SQLite is acceptable for simplicity if the architecture can reasonably migrate to PostgreSQL.

Using another equivalent technology is allowed if justified in the README.

Tests

Add a focused set of useful tests. Possible cases include:

- Valid CSV import
- Invalid or incomplete CSV structure
- Duplicate references
- Invalid date or currency
- Inconsistent amount calculation
- Successful correction and revalidation
- AI provider timeout or error
- Invalid structured response from the provider
- PDF extraction with missing required fields
- Bank statement producing multiple records

AI-assisted development

Claude Code, Codex, ChatGPT, GitHub Copilot and similar tools are allowed.

In the README, specify:

- Which tools were used
- How they were used
- Which generated parts required significant correction or redesign
- How you verified that you understood and tested the resulting code

You must be able to explain all submitted code during the follow-up interview.

Deliverables

Provide:

1. A Git repository link
2. A README containing:
 - Setup and run instructions
 - Environment variables
 - Main architecture and technical choices
 - Data model
 - AI provider integration approach
 - Assumptions
 - Completed and incomplete features
 - Known limitations
 - Production improvements
 - AI tools used
3. Tests
4. The provided sample files, or clear instructions on where to place them
5. Optional screenshots or a short demonstration video

Optional bonus items

These are not required:

- Docker Compose
- Background job processing for PDF extraction
- Idempotent import handling
- Duplicate-document detection
- Audit history for edits
- Pagination
- Multi-tenant isolation
- Authentication

- Deployment
- Provider fallback
- Cost or token usage tracking
- Confidence shown at field level

Prefer a coherent, reliable solution over an ambitious but incomplete one.

Evaluation

We will assess:

- Understanding and prioritization
- Backend structure and data modelling
- Validation and error handling
- PDF/AI connector design
- Frontend usability
- Tests
- Documentation
- Security basics
- Maintainability
- Ability to explain trade-offs

The follow-up discussion is part of the evaluation. We will ask you to explain the architecture, modify a business rule conceptually, discuss production concerns and walk through the AI extraction flow.

Good luck.